

Report on LoRa

Steps to connect LoRa to TTN and TTN to Firebase

By

Daram Pranay (1MS23EC033)

Supervisor

Dr. Sarvendranath Rimalapudi

Abstract

This project demonstrates how to connect a LoRa end device to The Things Network (TTN) and forward collected data to Firebase using Node.js. The workflow involves configuring a Dragino LoRaWAN gateway, registering the end device with TTN, formatting payloads, and creating a Node.js bridge to Firebase. The system successfully transmits test messages, establishing a foundation for IoT applications that require low-power, long-range communication and real-time cloud storage.

List of Contents

1. Introduction
2. Prerequisites
3. Connecting LoRa to TTN
4. Connecting TTN to Firebase
5. Result validation
6. Appendix

1.Introduction

The Internet of Things (IoT) requires efficient data collection and transfer from distributed devices to cloud platforms. LoRa (Long Range) technology provides low-power, long-range wireless communication, while TTN offers community-driven LoRaWAN infrastructure. Firebase provides real-time cloud storage and visualization.

This report focuses on building an end-to-end IoT communication pipeline using LoRa, TTN, and Firebase.

2. Prerequisites

- Hardware: Arduino Uno, LoRa module, Antenna, Dragino Gateway
- Software: Arduino IDE (LMIC library), Node.js, Firebase account, TTN account
- Accounts: The Things Network (TTN) Console, Firebase Console

3. Steps to Connect LoRa to TTN

- **Step 1:**
connect the LoRa module to Arduino Uno and attach an antenna to it fig 3.1
- **Step 2:**
On the power supply of Gateway (Dragino) with its antenna attached and the lights on gateway should look like this fig 3.2
- **Step 3:**
Connect your laptop to wifi (dragino-279fe4) of the gateway with password dragino+Dragino
- **Step 4:**
enter <http://10.130.1.1/> in the browser and login with these details
Username: root
Password: dragino
Make sure all the components have green tick fig 3.3



Fig 3.1 LoRa module on Arduino with antenna



Fig 3.2 Gateway with antenna

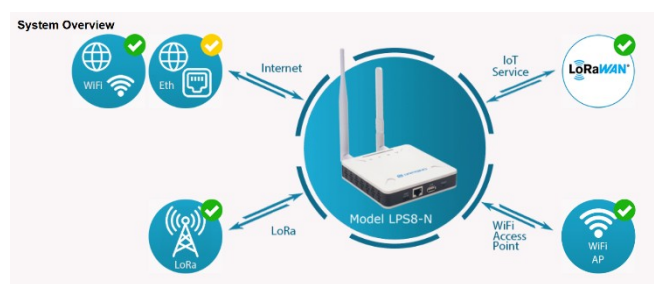


Fig 3.3 website after login

- **Step 5:**
In the website go to LoRaWAN and copy the Gateway EUI and make sure the server address is Australia (au1)
In the LoRa Configuration select the frequency Plan as IN865 refer fig 3.4 and fig 3.5
- **Step 6:**
Go to The Things Network (TTN) website and create a new account with region as Australia
- **Step 7:**
In Gateway column Register Gateway with a name and the Gateway EUI copied before and make sure it shows Gateway is connected refer fig 3.6

LoRaWAN Configuration

General Settings

Email:

Gateway EUI:

Primary LoRaWAN Server

Service Provider: Server Address:

Uplink Port: Downlink Port:

Fig 3.4 LoRaWAN Configuration

LoRa Configuration

Debug Level:

Radio Settings

Keep Alive Period (sec):

Frequency Plan:

Fig 3.5 LoRa Configuration

LPS8 n123
ID: indoor-lps8n12 [+ Add label](#) Last activity 25 seconds ago

General information

Gateway ID:

Gateway EUI:

Frequency plan:

Created at:

Gateway status ●

30 day uptime ⓘ

Only admins have access to uptime

Fig 3.6 TTN website After registering Gateway

- **Step 8:**
Create a new application with a unique name
- **Step 9:**
Register a new end device in the application enter details as per fig 3.7

- **Step 10:**
Select ABP and generate DevEUI, Device Address, AppSKey, NwkSkey, give it a name and register end device as shown in fig 3.8
- **Step 11:**
Write a code in Arduino IDE for sending hello message from LoRa to TTN install the required libraries

End device type

Input method ⓘ

☐ Select the end device in the LoRaWAN Device Repository

☒ Enter end device specifics manually

Frequency plan ⓘ *

India 865-867 MHz | v

LoRaWAN version ⓘ *

LoRaWAN Specification 1.0.3 | v

Regional Parameters version ⓘ *

RP001 Regional Parameters 1.0.3 revision A | v

Fig 3.7 End Device Configuration

Provisioning information

DevEUI ⓘ

70 B3 D5 7E D0 07 2A BB Generate 12/50 used

Device address ⓘ *

26 0D 87 CB Generate

AppSKey ⓘ *

B8 CE 2B 56 F0 36 F1 AF CD 0E 05 BA E5 D8 59 47 Generate

NwkSKey ⓘ *

79 11 09 D0 C4 56 10 F0 A8 A5 3C 2F 69 81 23 E0 Generate

End device ID ⓘ *

hello

Fig 3.8 During registering the device

- **Step 12:**
Copy AppSKey, NwkSkey and DevEUI of End Device and paste it in the code wherever required in the msb format shown in fig 3.9
- **Step 13:**
Select the board as Arduino UNO and port to which ever port LoRa is connected in Arduino IDE
- **Step 14:**
Upload the code it will be compiled and errors are shows fix the errors
- and upload again
- **Step 15:**
Open the serial monitor and select the time bound whichever is used in the code as you can see the hello message being sent fig 3.10

Device address

26 0D 53 A9

NwkSKey

0x7C, 0xD6, 0x15... msb ↔

AppSKey

0x83, 0x74, 0x59... msb ↔

Fig 3.9 msb format

- **Step 16:**
Go to TTN website you should see Messages appearing in the live data of the end device and also uplinks being sent in the Gateway live data
- **Step 17:**
The data appears in Hexadecimal form to convert it to ASCII format go to payload formatter and add a custom JavaScript formatter
add a code snippet to convert Hex to ASCII in JavaScript
- **Step 18:**
Now the Hello world message is clearly being transferred from LoRa to TTN

Formatter type*

Custom Javascript formatter

Formatter code*

```

1 function decodeUplink(input) {
2   // Convert bytes to ASCII string
3   let str = "";
4   for (let i = 0; i < input.bytes.length; i++) {
5     str += String.fromCharCode(input.bytes[i]);
6   }
7
8   return {
9     data: {
10      message: str    // store ASCII text in field "message"
11    }
12  };
13 }
14

```

Fig 3.11 Payloader in TTN

DevAddr: 26 8D AD C6	Rx1 Delay: 5			
DevAddr: 26 8D AD C6	Payload: { message: "Hello, world!" }	48 65 6C 6C 6F 2C 20 77 6F 72 6C 64 21	FPort: 1	Data rate: SF7BW125 SNR: 13.5 RSSI: -82
DevAddr: 26 8D AD C6				
DevAddr: 26 8D AD C6	Rx1 Delay: 5			
DevAddr: 26 8D AD C6	Payload: { message: "Hello, world!" }	48 65 6C 6C 6F 2C 20 77 6F 72 6C 64 21	FPort: 1	Data rate: SF7BW125 SNR: 14.2 RSSI: -82
DevAddr: 26 8D AD C6				
DevAddr: 26 8D AD C6	Rx1 Delay: 5			
DevAddr: 26 8D AD C6	Payload: { message: "Hello, world!" }	48 65 6C 6C 6F 2C 20 77 6F 72 6C 64 21	FPort: 1	Data rate: SF7BW125 SNR: 13.5 RSSI: -78
DevAddr: 26 8D AD C6				
DevAddr: 26 8D AD C6	Rx1 Delay: 5			
DevAddr: 26 8D AD C6	Payload: { message: "Hello, world!" }	48 65 6C 6C 6F 2C 20 77 6F 72 6C 64 21	FPort: 1	Data rate: SF7BW125 SNR: 13 RSSI: -61

Fig 3.12 Live data of end device

4. Steps to Transfer data from TTN to Firebase

- **Step 1:**
Install Node.js in your device and Go to firebase website and create a new firebase project
- **Step 2:**
Go to firebase database and create a database with nearby location in test mode
- **Step 3:**
In project settings go to service accounts and generate new private key in Node.js Configuration and save the file it will save in json configuration
- **Step 4:**
Create a file enter the code to connect from TTN to Firebase and save with .js extension

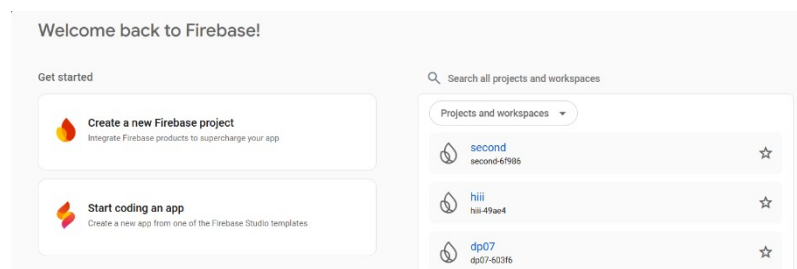


Fig 4.1 Firebase Website

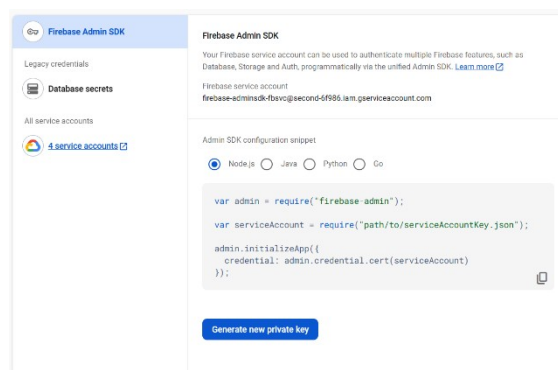


Fig 4.2 Generate new private key

- **Step 5:**
In TTN application go to API Keys and add an API Key you will get an API Key copy and keep it we will use it in coming steps
- **Step 6:**
In the code the enter the file name with which you stored the json file while loading your service account

- **Step 7:**
In place of username enter the Application ID from TTN which is present below the Application name
In place of password paste the API Key copied before
- **Step 8:**
In Client link make sure it is logged in with same server as in TTN (here au1)
In client.subscribe statement enter the APP ID

```
const mqtt = require('mqtt');
const admin = require('firebase-admin');

const serviceAccount = require('./firebase-key.json');//enter the file name with which you stored

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount)
});

const db = admin.firestore();

console.log(" Starting TTN → Firestore bridge...");

// Connect to TTN MQTT
const client = mqtt.connect('mqtt://au1.cloud.thethings.network:1883', {
  username: 'hello1234',//APP ID
  password: 'NNSXS.XVP43BOCT442F0GP7KGNPVC7UY262PT324DWAEI.TMKN3BIZRWOGTD6NO6WFJMF34H6C45LJZCLT2BID5CWYTT6RBA' // <-- Replace with
  your API key
});
```

Fig 4.3 Code after making the changes mentioned

- **Step 9:**
Make sure you are inside the folder through cd statement
Install firebase-admin into the folder through the below statement in terminal
npm install firebase-admin
- **Step 10:**
Install all the dependencies whichever are required (here firebase-admin is required)
- **Step 11:**
Enter the statement: node firebase-test.js
If the code is correct, it is executed

```
PS C:\Users\daram\Desktop\hello> npm install firebase-admin
up to date, audited 209 packages in 2s

23 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
PS C:\Users\daram\Desktop\hello> |
```

Fig 4.4 Terminal window after installing all dependencies

```
PS C:\Users\daram\Desktop\hello> node server.js
🚀 Starting TTN → Firestore bridge...
🔌 Connecting to TTN MQTT broker...
✅ Connected to TTN MQTT
```

- **Step 12:**

Make sure the data is sent from LoRa to TTN continuously meanwhile when the code is executed you can see that data is transferred to Firebase also which can be seen in the terminal you can see APP ID, RSSI, SF and many other data being transferred to firebase

- **Step 13:**

The data is obtained in Firebase Website whenever Data is transferred from LoRa to TTN, you can arrange the data according to the time and value of data also

```

✔ Saved data for hloooo to Firestore
❗ Raw MQTT message: {"end_device_ids":{"device_id":"hloooo","application_ids":{"application_id":"hello1234"},"dev_eui":"70B3D57ED0072A0D","dev_addr":"260D884D"},"correlation_ids":{"gs:uplink:01K3TQCF3P7FK7MNM57TBV7SF"},"received_at":"2025-08-29T10:53:43.622926842Z","uplink_message":{"f_port":1,"f_cnt":21,"frm_payload":"AB8=","decoded_payload":{"distance_cm":31},"rx_metadata":{"gateway_ids":{"gateway_id":"indoor-lps8n12","eui":"A84041FFFF279FE4"},"time":"2025-08-29T10:53:43.339260Z"},"timestamp":324513313,"rssi":-81,"channel_rssi":-81,"snr":13.2,"frequency_offset":"6283","uplink_token":"ChwKGgo0aW5kb29yLWxwczhuMTISCkhAQf//J5/kEKHc3poBGgwIt43GxQVQ/SyLxQEg60GA9LiDAg==","channel_index":2,"received_at":"2025-08-29T10:53:43.341697345Z"},"settings":{"data_rate":{"lora":{"bandwidth":125000,"spreading_factor":7,"coding_rate":"4/5"},"frequency":"865232500","timestamp":324513313,"time":"2025-08-29T10:53:43.339260Z"},"received_at":"2025-08-29T10:53:43.415620123Z"},"consumed_airtime":"0.046336s","packet_error_rate":"0.09090909","network_ids":{"net_id":"000013","ns_id":"EC656E0000000183"},"tenant_id":"ttn","cluster_id":"aui","cluster_address":"aui.cloud.thethings.network"}}}
📡 Data received from hloooo: { distance_cm: 31 }
✔ Saved data for hloooo to Firestore

```

Fig 4.6 Terminal window after execution of code

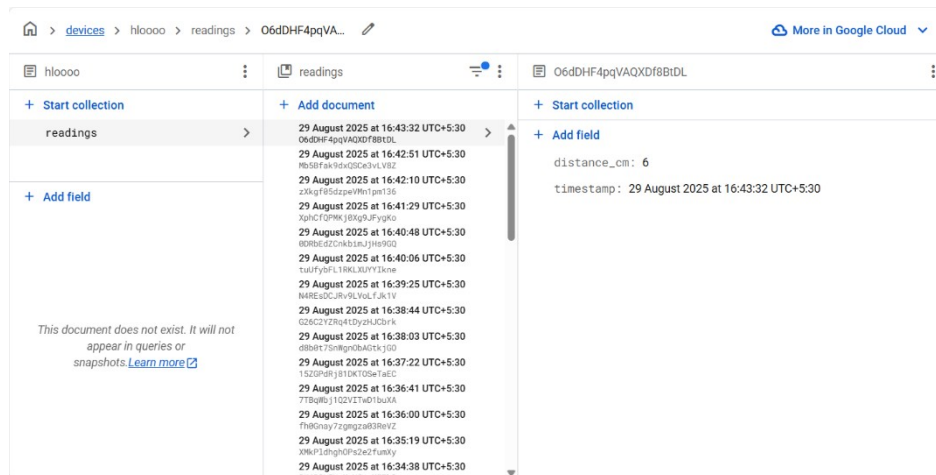


Fig 4.7 Firebase Window receiving data

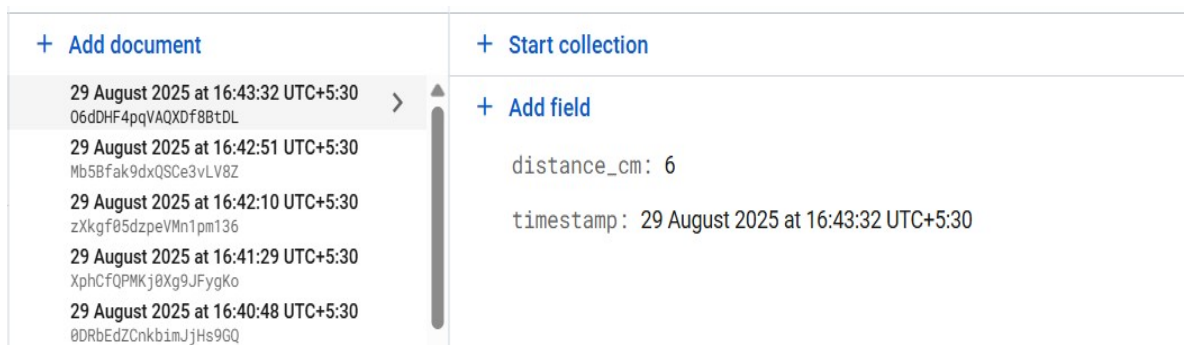
5. Firebase Data Validation

After running the Node.js script, check Firebase Console:

- Data entries should appear with timestamp and value.

Example JSON:

```
{  
  "timestamp": "2023-09-04T12:30:00Z",  
  "message": "Distance/Temperature/message"  
}
```



Troubleshooting

- Gateway not connecting? → Check EUI, server address, frequency plan.
- No uplinks in TTN? → Verify DevAddr, NwkSKey, AppSKey.
- Payload unreadable? → Update Payload Formatter (Hex → ASCII or sensor decoder).
- Firebase empty? → Check Node.js code, API key, MQTT subscription.

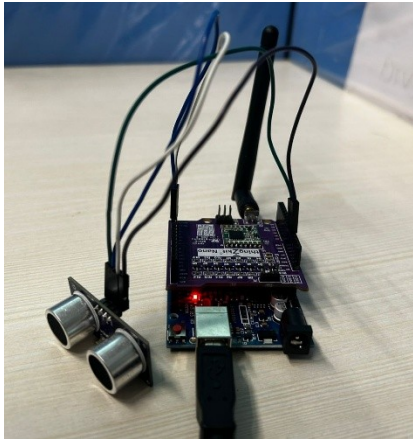


Fig: Integrated Ultrasonic Sensor

```

12:59:48.277 -> Sending distance: 9 cm
12:59:53.332 -> Sending distance: 9 cm
12:59:58.338 -> Sending distance: 9 cm
13:00:03.379 -> Sending distance: 10 cm
13:00:08.404 -> Sending distance: 9 cm
13:00:13.454 -> Sending distance: 9 cm
13:00:18.455 -> Sending distance: 9 cm
13:00:23.485 -> Sending distance: 9 cm
13:00:28.523 -> Sending distance: 10 cm
13:00:33.561 -> Sending distance: 17 cm

```

Fig: Distance readings on the Serial Monitor of Arduino IDE

sonic
ID: ultra-lora1

Last activity 24 seconds ago • 6 End devices

+ Add label

TIME	ENTITY ID	TYPE	DATA PREVIEW	Verbose stream	Export as JSON	Clear
16:55:58	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 6 } 00 06 FPort: 1 Data	☐	Export as JSON	Clear
16:55:22	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 36 } 00 24 FPort: 1 Data	☐	Export as JSON	Clear
16:54:46	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 40 } 00 28 FPort: 1 Data	☐	Export as JSON	Clear
16:54:10	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 40 } 00 28 FPort: 1 Data	☐	Export as JSON	Clear
16:53:34	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 40 } 00 28 FPort: 1 Data	☐	Export as JSON	Clear
16:52:58	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 39 } 00 27 FPort: 1 Data	☐	Export as JSON	Clear
16:52:22	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 40 } 00 28 FPort: 1 Data	☐	Export as JSON	Clear
16:51:45	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 39 } 00 27 FPort: 1 Data	☐	Export as JSON	Clear
16:51:09	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 39 } 00 27 FPort: 1 Data	☐	Export as JSON	Clear
16:50:33	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 39 } 00 27 FPort: 1 Data	☐	Export as JSON	Clear
16:49:57	lora-ultrasonic	Forward uplink data message	DevAddr: 26 00 F9 E0 Payload: { distance_cm: 40 } 00 28 FPort: 1 Data	☐	Export as JSON	Clear

Fig: Distance readings in live data of TTN

+ Add document

29 August 2025 at 16:43:32 UTC+5:30
06dDHF4pqVAQXDf8BtDL

29 August 2025 at 16:42:51 UTC+5:30
Mb5Bfak9dxQSCe3vLV8Z

29 August 2025 at 16:42:10 UTC+5:30
zXkgf05dzpeVMn1pm136

29 August 2025 at 16:41:29 UTC+5:30
XphCfQPMKj0Xg9JFygKo

29 August 2025 at 16:40:48 UTC+5:30
0DRbEdZCnkbimJjHs9GQ

29 August 2025 at 16:40:06 UTC+5:30
tuUfybFL1RKLXUYIknE

29 August 2025 at 16:39:25 UTC+5:30
N4REsDCJRv9LVofJk1V

29 August 2025 at 16:38:44 UTC+5:30
G26C2YZRq4tDyzHJCbrk

29 August 2025 at 16:38:03 UTC+5:30
d8b0t7SnWgnObAGtkjGO

+ Start collection

+ Add field

distance_cm: 6

timestamp: 29 August 2025 at 16:43:32 UTC+5:30

Fig: Distance readings in Firebase

6. Appendix

Example Arduino Code

```
#include <lmic.h>
#include <hal/hal.h>
static const u1_t NWKSKEY[16] = {...}; //Keys from TTN copied while registering
static const u1_t APPSKEY[16] = {...};
static const u4_t DEVADDR = ...;

void do_send(osjob_t* j) {
    static uint8_t msg[] = "Hello";
    LMIC_setTxData2(1, msg, sizeof(msg)-1, 0);
}
```

Example Node.js Code

```
const mqtt = require("mqtt");
const admin = require("firebase-admin");
const serviceAccount = require("./serviceAccountKey.json"); //name of JSON file with which
you saved

admin.initializeApp({
    credential: admin.credential.cert(serviceAccount),
    databaseURL: "https://<project-id>.firebaseio.com"
});

const client = mqtt.connect("mqtt://au1.cloud.thethings.network", {
    username: "AppID",
    password: "API_KEY"
});

client.on("connect", () => {
    client.subscribe("v3/AppID@ttn/devices/+/up");
});

client.on("message", (topic, message) => {
    const payload = JSON.parse(message.toString());
    admin.database().ref("uplinks").push({
        time: payload.received_at,
        data: payload.uplink_message.decoded_payload
    });
});
```